```
mov   rcx, 0x40      → 64 bits to take
                     is 0x40 → rcx
.loop:               (loop 16.4 times)
mov   rdx, rax       is rax → rdx
xor   rax, rbx       is rax = (rax | rbx)
and   rbx, rdx       is rbx = (rbx & rdx)
shl   rbx, 0x1       [0000 ... ...z]
loop  .loop          CF [0000 ... ...z0]
```

"shift logical left"

$\boxed{rax + rbx}$ — bit by bit ... rdx & rax

decrement rcx

rcx == 0 EXIT LOOP

∴ for (0x40 times)

{

rdx := rax

rax = (rax | rbx)

rbx = (rbx & rdx) = (rbx & rax)

rbx *= 2^1 = 2

ie rbx = (rbx & rax) / 2

}

ripple-carry, bit-by-bit, addition

ie $a+b = (a \oplus b) + 2(a \wedge b)$

```
mov   rcx, 0x40      (64 bits ...)
.loop:
mov   rdx, rax       (2 input registers)
xor   rax, rbx
and   rbx, rdx       (bit-by-bit on rbx)
shl   rbx, 0x1       → bit
loop  .loop
```

for (i=0; i<0x40; i++)

{

rdx = rax

rax = rax ⊕ rbx

rbx = rbx ∧ rdx = rax ∧ rbx

rbx = rbx · 2 = (rax ∧ rbx) · 2

}

$d = a_0$

$a_i := a_0 \oplus b_0$

$b_i := a_0 \wedge b_0$

$b_i := b_i \cdot 2$

$b = (a_0 \wedge b_0) \cdot 2$

for (0 → 40)

rdx = rax = $rax_{-1} \oplus rbx_{-1}$

rax = rax ⊕ rbx

rbx = (rbx ∧ rdx) · 2

$\Downarrow$

~~$d = a_0$~~

$a_1 = a_0 \oplus b_0$

same s'n $b = (a_0 \wedge b_0) \cdot 2$

$a+b = (a \oplus b) + 2(a \wedge b)$ ← "ripple-carry" addition

a same ~~temp~~ ~

same s'n

apply repeatedly bit-by-bit

a + b ?